

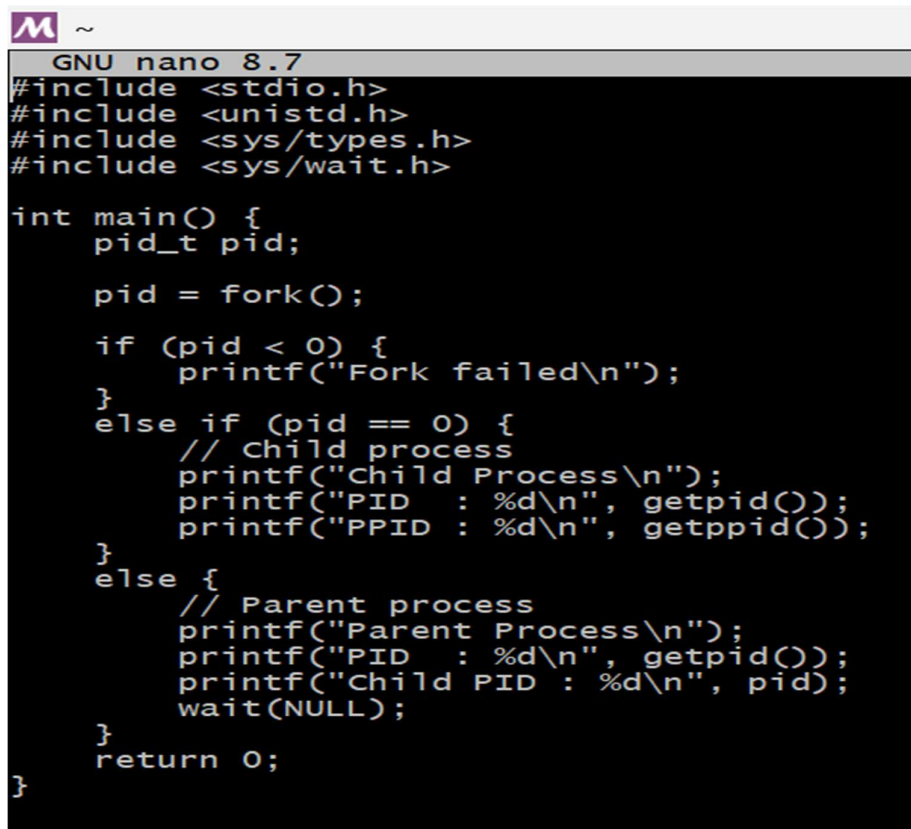
PRACTICAL 4:

Aim: Implement process creation Using the fork() system call in the Linux to generate parent and child processes, retrieve their PID and PPID, and analyse system behaviour by creating orphan and zombie processes.

1. Adam is working in an IT company. He has been given a task to reduce the load of a system by killing some of the processes running in the LINUX operating system. Which commands will he use to complete the given task with the help of the following operation?

- Kill processes by name.
- Kill a process based on the process name.
- Kill a single process at a time with the given process ID.

Code:



```
GNU nano 8.7
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
    }
    else if (pid == 0) {
        // Child process
        printf("Child Process\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());
    }
    else {
        // Parent process
        printf("Parent Process\n");
        printf("PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);
        wait(NULL);
    }
    return 0;
}
```

Output:

```
PRACHI@LAPTOP-CVJL7P0G MSYS ~
$ gcc --version
gcc (GCC) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

PRACHI@LAPTOP-CVJL7P0G MSYS ~
$ nano fork.c

PRACHI@LAPTOP-CVJL7P0G MSYS ~
$ gcc fork.c -o fork

PRACHI@LAPTOP-CVJL7P0G MSYS ~
$ ./fork
Child Process
Parent Process
PID : 560
PID : 559
PPID : 559
Child PID : 560

PRACHI@LAPTOP-CVJL7P0G MSYS ~
$
```

2. Write a program for process creation using C

- Orphan Process
- Zombie Process

Code:

```
M ~
GNU nano 8.7
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        sleep(5);
        printf("Child Process\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());
    }
    else {
        printf("Parent exiting\n");
    }
    return 0;
}
```

Output:

```
PRACHI@LAPTOP-CVJL7P0G MSYS ~
$ gcc --version
gcc (GCC) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

PRACHI@LAPTOP-CVJL7P0G MSYS ~
$ nano orphan.c

PRACHI@LAPTOP-CVJL7P0G MSYS ~
$ gcc orphan.c -o orphan

PRACHI@LAPTOP-CVJL7P0G MSYS ~
$ ./orphan
Parent exiting

PRACHI@LAPTOP-CVJL7P0G MSYS ~
$ Child Process
PID : 580
PPID : 1
|
```

3. Create the process using fork() system call.

- Child Process creation
- Parents process creation
- PPID and PID

Code:

```
M ~
GNU nano 8.7
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Child exiting\n");
    }
    else {
        sleep(10);
        printf("Parent running\n");
    }
    return 0;
}
```

Output:

```
PRACHI@LAPTOP-CVJL7P0G MSYS ~  
$ nano orphan.c  
  
PRACHI@LAPTOP-CVJL7P0G MSYS ~  
$ nano zombie.c  
  
PRACHI@LAPTOP-CVJL7P0G MSYS ~  
$ gcc zombie.c -o zombie  
  
PRACHI@LAPTOP-CVJL7P0G MSYS ~  
$ ./zombie  
Child exiting  
Parent running  
  
PRACHI@LAPTOP-CVJL7P0G MSYS ~  
$ ps -e1  
    PID    PPID    PGID    WINPID    TTY      UID     STIME  COMMAND  
    590     306     590     13828   pty0     197609  21:08:25 /usr/bin/ps  
    305       1     305     18760    ?        197609  20:29:18 /usr/bin/mintty  
    442     441     442     24892   pty1     197609  20:46:47 /usr/bin/bash  
    306     305     306     14252   pty0     197609  20:29:18 /usr/bin/bash  
    441       1     441     11148    ?        197609  20:46:47 /usr/bin/mintty
```